Modul Minggu 4 Sesi 3 — Flutter Widget UI Part 2

Mata Kuliah: Pengembangan Aplikasi Mobile => Interaktivitas, Form, dan Navigasi

Waktu	:	Jumat, 2 Oktober 2026
Materi	:	Flutter Widget UI part 2 — Interaktivitas, Form, dan Navigasi
Batas Pengumpulan	:	Minggu, 4 Oktober 2026, pukul 23.59 WIB
Tujuan	:	• Memahami perbedaan state yang berubah lewat setState() dan bagaimana UI merespons perubahan itu. • Mampu menerima dan memvalidasi input dari pengguna lewat TextField dan Form. • Mampu berpindah antar halaman (navigasi), termasuk mengirim dan menerima data antar halaman. • Mampu menampilkan Dialog dan SnackBar sebagai umpan balik ke pengguna. • Mampu membuat navigasi tab lewat BottomNavigationBar. • Mampu membuat widget kustom yang dapat dipakai ulang (reusable widget).
Pengampu	:	Tegar Arifin Prasetyo, S.Si., M.Si. Michael Joseph Christian Situmorang, S.Tr.Kom.

Pada sesi sebelumnya (W4S2), Anda sudah belajar widget-widget dasar yang sifatnya statis — tampil begitu aja tanpa merespons interaksi pengguna. Pada sesi ini, mahasiswa diberi tantang oleh TA :) Anda akan membuat widget yang benar-benar interaktif seperti merespons ketukan, menerima input teks, berpindah halaman, dan menampilkan data secara dinamis. Karena proyek Flutter dan cara menjalankannya sudah dibahas tuntas di W4S2, sesi ini langsung masuk ke materi, tanpa mengulang langkah persiapan proyek.

1. StatefulWidget dan setState — Widget yang Benar-benar Interaktif

Di W4S2, StatefulWidget sudah dipakai, tapi isinya belum pernah benar-benar "berubah" sebagai respons interaksi pengguna. Kuncinya ada di method setState(): setiap kali dipanggil, Flutter akan menjalankan ulang method build() dan menggambar ulang tampilan dengan nilai variabel yang terbaru.

Contoh berikut membuat counter sederhana: angka bertambah saat tombol + ditekan, berkurang saat tombol − ditekan, dan tidak bisa kurang dari 0. Buat file lib/widget_basic/counter_widget.dart:

```
import 'package:flutter/material.dart';

class CounterWidget extends StatefulWidget {
  const CounterWidget({super.key});

  @override
  State<CounterWidget> createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
  int _jumlah = 0;

  void _tambah() {
```

```
    setState(() {
      _jumlah++;
    });
  }

  void _kurang() {
    setState(() {
      if (_jumlah > 0) _jumlah--;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              '$_jumlah',
              style: const TextStyle(fontSize: 48, fontWeight: FontWeight.bold),
            ), // Text
            const SizedBox(height: 16),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                IconButton(
                  onPressed: _kurang,
                  icon: const Icon(Icons.remove_circle, size: 36),
                ), // IconButton
                const SizedBox(width: 24),
                IconButton(
                  onPressed: _tambah,
                  icon: const Icon(Icons.add_circle, size: 36),
                ), // IconButton
              ],
            ), // Row
          ],
        ), // Column
      ), // Center
    ); // Scaffold
  }
}
```

2. TextField — Menerima Input Teks

TextField adalah widget untuk menerima input teks dari pengguna (seperti kolom pencarian atau form login). Untuk membaca nilai yang diketik, Anda butuh TextEditingController yang dipasangkan ke properti controller pada TextField.

Contoh berikut menampilkan kembali teks yang diketik pengguna secara langsung (real-time) di bawah kolom input. Buat file lib/widget_basic/textfield_widget.dart:

```dart
import 'package:flutter/material.dart';

class TextFieldWidget extends StatefulWidget {
  const TextFieldWidget({super.key});

  @override
  State<TextFieldWidget> createState() => _TextFieldWidgetState();
}

class _TextFieldWidgetState extends State<TextFieldWidget> {
  final TextEditingController _namaController = TextEditingController();
  String _preview = '';

  @override
  void dispose() {
    _namaController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('TextField')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            TextField(
              controller: _namaController,
              decoration: const InputDecoration(
                labelText: 'Nama Anda',
                border: OutlineInputBorder(),
                prefixIcon: Icon(Icons.person),
              ), // InputDecoration
              onChanged: (value) {
                setState(() {
                  _preview = value;
                });
              },
            ), // TextField
            const SizedBox(height: 16),
            Text('Halo, $_preview!'),
          ],
        ), // Column
      ), // Padding
    ); // Scaffold
  }
}
```

3. Form dan Validasi

Untuk input yang lebih dari satu kolom dan perlu divalidasi (misalnya tidak boleh kosong, harus berupa angka, dan sebagainya), Flutter menyediakan widget Form yang membungkus beberapa TextFormField sekaligus. Form dikontrol lewat GlobalKey<FormState>, dan validasi dijalankan lewat formKey.currentState!.validate().

Contoh berikut membuat form sederhana dengan dua kolom (nama dan email), masing-masing dengan aturan validasi. Buat file lib/widget_basic/form_widget.dart:

```dart
import 'package:flutter/material.dart';

class FormWidget extends StatefulWidget {
  const FormWidget({super.key});

  @override
  State<FormWidget> createState() => _FormWidgetState();
}

class _FormWidgetState extends State<FormWidget> {
  final _formKey = GlobalKey<FormState>();
  final _namaController = TextEditingController();
  final _emailController = TextEditingController();

  void _submit() {
    if (_formKey.currentState!.validate()) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Form valid! Nama: ${_namaController.text}')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Form Pendaftaran')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Form(
          key: _formKey,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              TextFormField(
                controller: _namaController,
                decoration: const InputDecoration(
                  labelText: 'Nama Lengkap',
                  border: OutlineInputBorder(),
                ), // InputDecoration
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Nama wajib diisi';
                  }
                  return null;
                },
              ), // TextFormField
              const SizedBox(height: 16),
              TextFormField(
                controller: _emailController,
                decoration: const InputDecoration(
                  labelText: 'Email',
                  border: OutlineInputBorder(),
                ), // InputDecoration
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Email wajib diisi';
                  }
                  if (!value.contains('@')) {
                    return 'Format email tidak valid';
```

```
                }
                return null;
              },
            ), // TextFormField
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: _submit,
              child: const Text('Daftar'),
            ), // ElevatedButton
          ],
        ), // Column
      ), // Form
    ), // Padding
  ); // Scaffold
  }
}
```

4. Navigasi Antar Halaman

Hampir semua aplikasi nyata punya lebih dari satu halaman. Flutter mengelola perpindahan antar halaman lewat Navigator, yang bekerja seperti tumpukan kartu (stack): Navigator.push() menambahkan halaman baru di atas, Navigator.pop() mengambil halaman itu kembali (biasanya saat tombol kembali ditekan).

4.1 Navigasi dasar dan kirim data ke halaman tujuan

Buat dua file baru: lib/pages/produk_list_page.dart (halaman daftar) dan lib/pages/produk_detail_page.dart (halaman detail). Mulai dari halaman detail, karena lebih sederhana:

```
import 'package:flutter/material.dart';

class ProdukDetailPage extends StatelessWidget {
  final String namaProduk;
  final int harga;

  const ProdukDetailPage({
    super.key,
    required this.namaProduk,
    required this.harga,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Detail Produk')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              namaProduk,
              style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
            ), // Text
            const SizedBox(height: 8),
            Text('Rp $harga'),
```

```
          ],
        ), // Column
      ), // Padding
    ); // Scaffold
  }
}
```

Lalu halaman daftar — saat salah satu item di-tap, data namaProduk dan harga dikirim ke ProdukDetailPage lewat parameter constructor-nya:

```
import 'package:flutter/material.dart';
import 'package:nama_proyek_anda/pages/produk_detail_page.dart';

class ProdukListPage extends StatelessWidget {
  ProdukListPage({super.key});

  final List<Map<String, dynamic>> produk = [
    {'nama': 'Kursi Minimalis', 'harga': 350000},
    {'nama': 'Meja Kerja Kayu', 'harga': 750000},
    {'nama': 'Lampu Meja LED', 'harga': 120000},
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Produk')),
      body: ListView.builder(
        itemCount: produk.length,
        itemBuilder: (context, index) {
          final item = produk[index];
          return ListTile(
            title: Text(item['nama']),
            subtitle: Text('Rp ${item['harga']}'),
            trailing: const Icon(Icons.chevron_right),
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => ProdukDetailPage(
                    namaProduk: item['nama'],
                    harga: item['harga'],
                  ),
                ), // MaterialPageRoute
              ); // Navigator.push
            },
          ); // ListTile
        },
      ), // ListView.builder
    ); // Scaffold
  }
}
```

4.2 Mengambil data balik dari halaman yang dibuka (await Navigator.push)

Selain mengirim data ke halaman tujuan, Anda juga bisa menerima data balik dari halaman itu — misalnya halaman detail punya tombol "Pilih" yang mengembalikan nama produk ke halaman sebelumnya. Tambahkan tombol berikut di body ProdukDetailPage :

```
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context, namaProduk);
              },
              child: const Text('Pilih Produk Ini'),
            ), // ElevatedButton
```

Lalu ubah cara memanggil Navigator.push di ProdukListPage supaya menunggu (await) nilai baliknya:

```
            onTap: () async {
              final hasilPilihan = await Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => ProdukDetailPage(
                    namaProduk: item['nama'],
                    harga: item['harga'],
                  ),
                ), // MaterialPageRoute
              ); // Navigator.push

              if (hasilPilihan != null) {
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text('Anda memilih: $hasilPilihan')),
                );
              }
            },
```

5. Dialog dan SnackBar

Selain SnackBar (notifikasi kecil di bagian bawah layar, sudah dipakai di bagian 3 dan 4), Flutter juga punya AlertDialog — jendela kecil di tengah layar yang biasa dipakai untuk konfirmasi penting (misalnya konfirmasi hapus data).

Contoh berikut menampilkan dialog konfirmasi sebelum benar-benar "menghapus" sesuatu. Buat file lib/widget_basic/dialog_widget.dart:

```
import 'package:flutter/material.dart';

class DialogWidget extends StatelessWidget {
  const DialogWidget({super.key});

  void _tampilkanKonfirmasi(BuildContext context) {
    showDialog(
      context: context,
      builder: (context) {
        return AlertDialog(
          title: const Text('Hapus Item?'),
          content: const Text('Item ini akan dihapus secara permanen.'),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Batal'),
```

```
          ), // TextButton
          TextButton(
            onPressed: () {
              Navigator.pop(context);
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Item telah dihapus')),
              );
            },
            child: const Text('Hapus'),
          ), // TextButton
        ],
      ); // AlertDialog
    },
  ); // showDialog
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Dialog')),
    body: Center(
      child: ElevatedButton(
        onPressed: () => _tampilkanKonfirmasi(context),
        style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
        child: const Text('Hapus Item'),
      ), // ElevatedButton
    ), // Center
  ); // Scaffold
 }
}
```

6. BottomNavigationBar — Navigasi Antar Tab

BottomNavigationBar menampilkan beberapa tab di bagian bawah layar (seperti pada aplikasi Instagram atau Tokopedia), memungkinkan pengguna berpindah antar "halaman utama" tanpa animasi push/pop seperti di Bagian 4 — cukup dengan mengganti body yang ditampilkan berdasarkan tab yang aktif.

Buat file lib/pages/main_navigation_page.dart:

```
import 'package:flutter/material.dart';

class MainNavigationPage extends StatefulWidget {
 const MainNavigationPage({super.key});

 @override
 State<MainNavigationPage> createState() => _MainNavigationPageState();
}

class _MainNavigationPageState extends State<MainNavigationPage> {
 int _tabTerpilih = 0;

 final List<Widget> _halaman = const [
   Center(child: Text('Halaman Beranda', style: TextStyle(fontSize: 20))),
   Center(child: Text('Halaman Favorit', style: TextStyle(fontSize: 20))),
   Center(child: Text('Halaman Profil', style: TextStyle(fontSize: 20))),
```

```
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Main Navigation')),
      body: _halaman[_tabTerpilih],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _tabTerpilih,
        onTap: (index) {
          setState(() {
            _tabTerpilih = index;
          });
        },
        items: const [
          BottomNavigationBarItem(
            icon: Icon(Icons.home),
            label: 'Beranda',
          ), // BottomNavigationBarItem
          BottomNavigationBarItem(
            icon: Icon(Icons.favorite),
            label: 'Favorit',
          ), // BottomNavigationBarItem
          BottomNavigationBarItem(
            icon: Icon(Icons.person),
            label: 'Profil',
          ), // BottomNavigationBarItem
        ],
      ), // BottomNavigationBar
    ); // Scaffold
  }
}
```

7. Widget Kustom (Reusable Widget)

Semakin kompleks aplikasi Anda, semakin penting untuk tidak menulis ulang kode UI yang sama berkali-kali. Solusinya: bungkus kombinasi widget yang sering dipakai ke dalam class widget Anda sendiri, lalu beri parameter supaya bisa dipakai ulang dengan data yang berbeda-beda — persis seperti membuat class Barang di modul OOP, tapi sekarang class-nya adalah sebuah widget.

Contoh berikut membungkus tampilan kartu produk (gabungan dari Container, Column, Text — mirip pola di W4S2) menjadi satu widget ProductCard yang menerima nama dan harga sebagai parameter. Buat file lib/widget_basic/product_card.dart:

```
import 'package:flutter/material.dart';

class ProductCard extends StatelessWidget {
  final String nama;
  final int harga;
  final VoidCallback onTap;

  const ProductCard({
    super.key,
    required this.nama,
    required this.harga,
    required this.onTap,
```

```
  });

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: Colors.grey[200],
          borderRadius: BorderRadius.circular(12),
        ), // BoxDecoration
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              nama,
              style: const TextStyle(fontWeight: FontWeight.w600, fontSize: 16),
            ), // Text
            const SizedBox(height: 4),
            Text(
              'Rp $harga',
              style: const TextStyle(color: Colors.grey),
            ), // Text
          ],
        ), // Column
      ), // Container
    ); // GestureDetector
  }
}
```

Widget ProductCard ini sekarang bisa dipakai berkali-kali di halaman mana pun, cukup dengan mengganti parameternya, misalnya:

```
ProductCard(
  nama: 'Kursi Minimalis',
  harga: 350000,
  onTap: () {
    print('Kursi Minimalis ditekan');
  },
),
```

Tugas — Studi Kasus: Mini Aplikasi Katalog Produk

Gabungkan semua yang sudah dipelajari di W4S2 dan W4S3 menjadi satu aplikasi kecil yang utuh. Buat aplikasi katalog produk sederhana dengan ketentuan berikut:

- Halaman Daftar Produk — menampilkan produk dalam bentuk ListView atau GridView, memakai widget ProductCard (Bagian 7) yang Anda buat sendiri, berisi minimal 5 produk berbeda.
- Halaman Detail Produk — terbuka saat salah satu produk di-tap (Bagian 4), menampilkan nama, harga, dan gambar/ikon produk, serta tombol "Tambah ke Keranjang".
- State jumlah keranjang — gunakan setState (Bagian 1) untuk menghitung jumlah item yang sudah ditambahkan ke keranjang, ditampilkan sebagai badge angka di ikon keranjang pada AppBar (lihat kembali contoh Stack + Positioned di modul W4S2 Bagian 2.11).

- Form Tambah Produk — buat halaman baru dengan Form (Bagian 3) berisi minimal 2 kolom (nama produk, harga) lengkap dengan validasi, untuk menambahkan produk baru ke daftar.
- Konfirmasi dan umpan balik — tampilkan AlertDialog (Bagian 5) saat pengguna ingin menghapus produk dari daftar, dan SnackBar sebagai konfirmasi setelah aksi tambah/hapus berhasil.
- Navigasi tab — bungkus halaman Daftar Produk dan halaman Profil (boleh halaman kosong/sederhana) ke dalam BottomNavigationBar (Bagian 6).

Langkah pengumpulan:

- Rekam video demo aplikasi berjalan (tunjukkan semua fitur di atas) sekaligus jelaskan bagian kode yang menerapkan tiap konsepnya.
- Video berdurasi 10 menit paling lama dan memiliki vocal yang jelas.
- Unggah video ke YouTube atau Google Drive, lalu kumpulkan link-nya di ecourse dalam file .txt.
- Untuk keseluruhan kodenya, masukkan ke Github masing masing, kemudian salin link nya.
- Link video dan kodenya disimpan ke dalam bentuk txt dengan nama file: **NIM_Nama_TugasW4S3.txt**.